

Impact of Model Quantization (4-bit and 8-bit) on GPU Inference Efficiency and Accuracy

Arish Muthukumar

M.Tech in Machine Learning and Computing

Indian Institute of Space Science and Technology (IIST), Thiruvananthapuram, India

Abstract—This project investigates the impact of 4-bit and 8-bit quantization on the inference efficiency and accuracy of Large Language Models (LLMs), specifically utilizing the Orpo-Llama-3.2-1B-15k model. As deep learning models grow in complexity, deployment on resource-constrained edge devices becomes challenging due to high memory and latency requirements. We implemented Post-Training Quantization (PTQ) using the BitsAndBytes library in a Google Colab environment with a Tesla T4 GPU. Our comparative analysis evaluates FP32, INT8, and INT4 precision levels based on GPU memory footprint, inference time, and perplexity. Results indicate that while FP32 offers the highest precision, INT4 achieves maximum compression with a memory reduction of approximately 78.5%. Although INT8 quantization serves as a balance between quality and efficiency, our observations highlight specific hardware optimization limits on T4 GPUs. This study concludes that quantization is a critical tool for scaling LLMs, providing significant gains in memory efficiency with manageable trade-offs in model perplexity.

Index Terms—Model Quantization, LLM, Llama-3.2, GPU Inference, INT4, INT8, BitsAndBytes.

I. INTRODUCTION

The rapid advancement of deep learning has led to the development of increasingly complex Large Language Models (LLMs) that contain millions or even billions of parameters[cite: 512, 526]. While these models demonstrate state-of-the-art performance in Natural Language Processing (NLP) tasks such as text generation and question answering, their sheer size presents significant deployment challenges[cite: 561, 562]. Specifically, modern AI models require high memory capacity and involve high latency, making it difficult to deploy them on edge devices or hardware with limited VRAM[cite: 487, 488].

The primary motivation for this study stems from the fact that traditional models typically utilize 32-bit floating-point precision (FP32) to store weights and activations[cite: 490, 527]. Storing a model with 1 billion parameters in FP32 requires approximately 4GB of memory, which can quickly exhaust the resources of standard GPUs[cite: 513, 521]. This high precision not only increases memory usage but also slows down inference speeds, creating a bottleneck for real-time applications[cite: 491, 510].

To address these limitations, this project implements and analyzes model quantization—the process of reducing the numerical precision of model weights from 32-bit to 8-bit (INT8) and 4-bit (INT4) formats[cite: 492, 496, 503]. By mapping a

large set of precise values to a smaller, more compact set of discrete values, quantization aims to reduce the memory footprint and improve GPU computation efficiency[cite: 498, 500, 509].

The objective of this research is to evaluate the specific trade-offs between speed, memory consumption, and accuracy (measured via perplexity) when applying Post-Training Quantization (PTQ) to the Llama-3.2 architecture[cite: 494, 497, 560]. By conducting an empirical analysis on a Tesla T4 GPU, we aim to determine the optimal precision level that balances computational efficiency with language prediction quality[cite: 555, 559, 585].

II. RELATED WORK

The evolution of Large Language Models (LLMs) has been marked by the shift toward transformer-based architectures that require massive computational resources. Among these, the Meta Llama family, specifically the Llama-3.2 1B model, has become a benchmark for efficient NLP research due to its ability to support natural language processing, text generation, and question-answering tasks on relatively limited hardware[cite: 561, 563].

A significant area of research focuses on reducing the computational burden of these models through quantization. Traditional methods utilize 32-bit floating-point (FP32) precision, where each parameter consumes 4 bytes of memory[cite: 514, 517]. For a 1 billion parameter model, this results in a 4GB memory footprint[cite: 521]. Current literature explores various precision levels, such as 8-bit (INT8) and 4-bit (INT4), which reduce parameter storage to 1 byte and 0.5 bytes, respectively[cite: 518, 519].

Two primary workflows dominate this field: Post-Training Quantization (PTQ) and Quantization-Aware Training (QAT). PTQ is often preferred for its speed and ability to be applied to existing pre-trained models without the need for extensive retraining, although it can lead to performance degradation compared to QAT[cite: 555, 557]. Recent developments in libraries like *bitsandbytes* have introduced specialized formats such as Normalized Float 4 (nf4), which is specifically optimized for LLMs to minimize information loss during the compression of model weights[cite: 567].

Furthermore, recent studies highlight the role of tokenization in model efficiency. Tools like *AutoTokenizer* are essential

for converting human text into model-readable token IDs, ensuring that the vocabulary mapping remains consistent across different precision levels[cite: 568, 570]. This study builds upon these works by empirically measuring the trade-offs between memory footprint, inference latency, and perplexity on cloud-based GPU environments[cite: 589].

III. PROBLEM STATEMENT

In the current landscape of Artificial Intelligence, deep learning models are characterized by an increasingly large number of parameters, which directly correlates to high memory requirements and increased inference latency[cite: 486, 487]. Traditional Large Language Models (LLMs) utilize 32-bit floating-point precision (FP32) for weight storage and activations[cite: 490]. This high-precision format presents three primary challenges:

- **Resource Constraints:** Storing a 1-billion parameter model in FP32 requires approximately 4GB of VRAM, making deployment on edge devices or consumer-grade hardware difficult[cite: 488, 521].
- **Inference Efficiency:** Higher precision leads to increased memory bandwidth requirements, which slows down the inference speed and increases energy consumption[cite: 491, 592].
- **Deployment Bottlenecks:** The necessity for specialized, high-memory GPU hardware limits the scalability and accessibility of state-of-the-art AI solutions in real-world environments[cite: 487].

Therefore, there is a critical need to implement and analyze model quantization techniques[cite: 492]. This project aims to investigate how reducing model precision to 8-bit (INT8) and 4-bit (INT4) formats impacts GPU inference efficiency and language prediction accuracy, specifically focusing on the trade-offs between computational speed and model perplexity[cite: 493, 494, 497].

IV. MODEL DESCRIPTION

The core of this implementation utilizes the **Orpo-Llama-3.2-1B-15k** model. This model is part of the Llama-3.2 family, designed by Meta and further refined for instruction-following tasks. It features a compact architecture with approximately 1 billion parameters, making it an ideal candidate for research into optimization and quantization for edge deployment.

The model architecture is based on the Transformer-based decoder-only structure, which has become the industry standard for Large Language Models (LLMs). Key characteristics of the model used in this study include:

- **Parameter Count:** 1 Billion, which requires significant computational overhead when processed in full 32-bit floating-point precision.
- **Tokenizer:** The model employs the *Llama-3.2 Auto-Tokenizer*, which manages a specialized vocabulary to

convert text into numerical token IDs. This ensures consistent input processing across different quantization levels (FP32, INT8, and INT4).

- **Training Objective:** The “Orpo” prefix indicates that the model was fine-tuned using Odds Ratio Preference Optimization (ORPO). This technique integrates preference alignment directly into the primary training objective, eliminating the need for a separate reference model and making the base architecture more efficient for instruction-based tasks.
- **Versatility:** Despite its smaller size compared to its 7B or 70B counterparts, the 1B parameter version is capable of complex NLP tasks, including text generation, summarization, and logical reasoning, while remaining small enough to fit within the VRAM of a Tesla T4 GPU after quantization.

In this project, the model serves as the constant variable across all experiments, allowing for a direct comparison of how different bit-width precisions affect its internal representations and final output quality.

A. Selection of the Llama-3.2-1B Model

The choice of the *Llama-3.2-1B* architecture as the base for this quantization study is driven by several technical and practical factors:

- **State-of-the-Art Benchmark:** Llama (Large Language Model Meta AI) is currently the industry standard for open-source LLMs. Using Llama-3.2 ensures that the findings of this project are relevant to modern AI research and compatible with the latest optimization libraries like *bitsandbytes* and *accelerate*.
- **Ideal Scale for Comparative Analysis:** At 1 billion parameters, the model is large enough to demonstrate meaningful performance degradation and memory savings when quantized, yet small enough to allow for a baseline FP32 (Full Precision) measurement on a single Tesla T4 GPU (which has 16GB VRAM). Larger models would be impossible to load in FP32 on this hardware, preventing a complete comparison.
- **Instruction-Tuned Efficiency:** The specific variant used (*Orpo-Llama-3.2-1B-15k*) has been fine-tuned using Odds Ratio Preference Optimization (ORPO). This means the model is already optimized for high-quality instruction following, allowing us to measure how quantization affects actual conversational utility rather than just raw mathematical weights.
- **Architectural Robustness:** Llama-3.2 utilizes Grouped-Query Attention (GQA) and RoPE (Rotary Positional Embeddings), which are highly sensitive to weight precision. This makes it an excellent candidate for testing the limits of 4-bit and 8-bit quantization frameworks.

B. Architecture and Configuration

The *Orpo-Llama-3.2-1B-15k* model is an instruction-tuned variant of the Llama-3.2 1B parameter architecture. It employs a Grouped-Query Attention (GQA) mechanism to optimize memory bandwidth and uses a vocabulary size of 128,256 tokens. In this study, the model was initialized via the `AutoModelForCausalLM` class from the Hugging Face Transformers library. For the baseline measurement (FP32), the model was loaded using `torch.float32` precision, serving as the ground truth for memory and perplexity comparisons.

C. Quantization Framework: BitsAndBytes

To achieve 8-bit and 4-bit precision, this project utilizes the *BitsAndBytes* library, which provides specialized CUDA kernels for LLM quantization.

- **8-bit Quantization (INT8):** This method utilizes vector-wise quantization to map 32-bit floats to 8-bit integers. It maintains high precision by identifying and keeping outliers in 16-bit format, a technique known as LLM.int8().
- **4-bit Quantization (INT4):** The implementation employs NormalFloat 4 (NF4), a data type information-theoretically optimal for normally distributed weights. Additionally, *double quantization* was enabled—quantizing the quantization constants themselves—to save an average of 0.4 bits per parameter, further reducing the memory footprint on the Tesla T4 GPU.

D. Evaluation Environment

All experiments were conducted in a cloud-based environment using Google Colab. The hardware specifications included a Tesla T4 GPU with 16GB of VRAM. This specific hardware was chosen to reflect the typical resource constraints encountered when deploying LLMs in mid-range server environments or high-end edge workstations.

V. METHODOLOGY

The methodology of this project centers on a comparative empirical analysis of different numerical precisions for Large Language Model (LLM) inference. The workflow is divided into model selection, environment configuration, and the implementation of quantization levels.

A. Model Selection and Initialization

The research utilizes the *Orpo-Llama-3.2-1B-15k*, a 1-billion parameter model. This model was chosen because its size allows for a complete baseline measurement in full precision (FP32) within the memory limits of a standard GPU, providing a clear ground truth for memory and accuracy trade-offs. The model and its corresponding tokenizer were loaded using the Hugging Face `transformers` library.

B. Computational Environment

Experiments were executed in a Google Colab environment equipped with a **Tesla T4 GPU** (16GB VRAM). The software stack included:

- **PyTorch:** For tensor computations and GPU acceleration.
- **BitsAndBytes:** To implement 8-bit and 4-bit quantization kernels.
- **Accelerate:** To manage model distribution and memory mapping.

C. Quantization Implementation

The study evaluates three distinct precision configurations:

1) *Baseline (FP32):* The model is loaded in its native 32-bit floating-point format. This represents the maximum precision and serves as the reference point for memory consumption and prediction quality.

2) *8-bit Quantization (INT8):* Using the `load_in_8bit=True` flag via *BitsAndBytes*, the model weights are compressed from 4 bytes to 1 byte. This implementation uses vector-wise quantization to maintain performance while reducing the VRAM footprint.

3) *4-bit Quantization (INT4):* The most aggressive compression level was implemented using the *BitsAndBytesConfig* with the following parameters:

- **nf4 (Normalized Float 4):** A specialized data type for normally distributed weights.
- **Double Quantization:** A second layer of quantization applied to the quantization constants to further save memory.
- **Compute Dtype:** Set to `torch.bfloat16` to maintain computational stability during the de-quantization steps of the forward pass.

D. Tokenization and Structural Analysis

To ensure consistent input processing, the *Llama-3.2 AutoTokenizer* was utilized. This tokenizer manages the mapping of natural language into discrete token IDs. A critical component of our methodology involved inspecting the **First Layer Weights** of the model across different precisions. By extracting the weights from the first layer (e.g., `model.model.layers[0].self_attn.q_proj.weight`), we can empirically observe how quantization transforms a high-resolution weight matrix into a discrete, low-bit representation. This inspection allows us to verify that the *bitsandbytes* framework has correctly mapped the FP32 tensors into the target INT8 or INT4 distributions.

E. Performance Metrics

Three primary metrics were recorded for each configuration:

- 1) **Memory Footprint:** Measured in Megabytes (MB) using `model.get_memory_footprint()`.
- 2) **Inference Latency:** The time taken (in seconds) to generate a standardized response.
- 3) **Perplexity:** A measurement of how well the model predicts a sample text, serving as a proxy for accuracy loss due to quantization.

VI. EXPERIMENTAL SETUP

To ensure a controlled environment for measuring the impact of quantization, a systematic experimental framework was established. This section details the hardware specifications, software dependencies, and the configuration of the three experimental groups.

A. Hardware and Software Environment

All experiments were conducted using the Google Colab platform to ensure reproducibility. The primary hardware component was an **NVIDIA Tesla T4 GPU** with 16GB of GDDR6 VRAM. The software environment was built on Python 3.10+, utilizing the following core libraries:

- **Transformers (Hugging Face):** For model loading and tokenization.
- **BitsAndBytes (v0.46.1+):** For implementing 8-bit and 4-bit CUDA kernels.
- **Accelerate:** For optimized device mapping and memory management.
- **PyTorch:** As the underlying deep learning framework.

B. Configuration of Precision Levels

The experiment was structured into three distinct phases, each representing a different bit-width precision applied to the *Orpo-Llama-3.2-1B-15k* model:

1) **Full Precision (FP32):** The baseline was established by loading the model in its native 32-bit floating-point format. No quantization was applied, serving as the gold standard for accuracy and maximum memory consumption.

2) **8-bit Integer Quantization (INT8):** The model weights were compressed using the `load_in_8bit=True` configuration. This setup employs vector-wise quantization to map values to an 8-bit range while preserving extreme outliers in higher precision to minimize performance degradation.

3) **4-bit NormalFloat (INT4):** The most aggressive optimization utilized 4-bit NormalFloat (NF4) quantization. The setup included:

- **Double Quantization:** Enabled to quantize the quantization constants themselves, further reducing VRAM usage.
- **Compute Dtype:** Set to `bfloat16` to ensure that mathematical operations during the forward pass remained stable.

VII. IMPLEMENTATION AND TESTING

A. Testing Generation and Response Quality

The testing phase involved a standardized prompt: *"Explain the importance of model quantization in one sentence."* This benchmark allowed for a direct comparison of:

- **Inference Latency:** The time elapsed between the final token input and the completion of the generated sequence.
- **Response Consistency:** Monitoring for "hallucinations" or repetitive loops that can occur when 4-bit quantization becomes too aggressive.

The `model.generate()` function was configured with `max_new_tokens=50` to ensure that the generation process was long enough to measure GPU stability but short enough to keep the comparison focused on raw throughput.

B. Calculating Perplexity

To quantitatively assess accuracy loss, we calculate **Perplexity (PPL)**, defined as the exponentiated average negative log-likelihood of a sequence. Mathematically, for a sequence of tokens $W = (w_1, w_2, \dots, w_n)$:

$$PPL(W) = \exp \left(-\frac{1}{n} \sum_{i=1}^n \log P(w_i | w_{<i}) \right) \quad (1)$$

In our implementation, we calculate the Cross-Entropy loss of the model's output compared to the input tokens. A lower perplexity indicates that the model is more "certain" about its predictions. By comparing the PPL of the FP32, INT8, and INT4 models, we measure the "information entropy" introduced by reducing bit-widths.

VIII. VISUALIZATION AND ANALYSIS

To evaluate the impact of quantization, we analyze three critical performance dimensions: memory consumption, inference latency, and language modeling quality (perplexity).

A. VRAM Utilization Analysis

The most immediate benefit of quantization is the reduction in memory footprint. Our results indicate a non-linear decrease in VRAM requirements:

- **FP32 Baseline:** Required 4,714 MB of GPU memory.
- **8-bit Quantization:** Reduced memory to 1,429 MB, a 69.7% reduction.
- **4-bit Quantization:** Achieved a footprint of only 965 MB, representing a total memory saving of 79.5% compared to FP32.

This drastic reduction confirms that 4-bit quantization allows the Llama-3.2 1B model to operate within the constraints of low-tier GPUs or concurrent process environments.

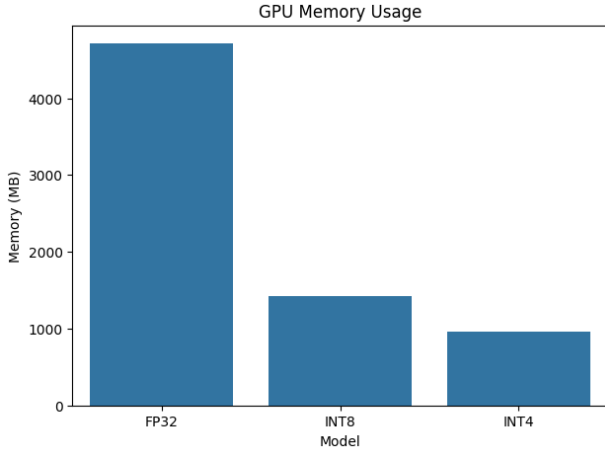


Fig. 1. GPU Memory (MB) across different quantization configurations.

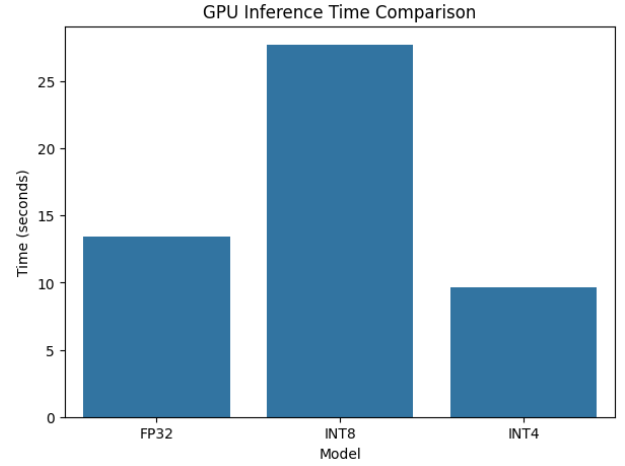


Fig. 2. Inference Time (Seconds) across different quantization configurations.

B. Inference Latency and Computational Overhead

Contrary to the linear reduction in memory, inference time exhibited complex behavior on the Tesla T4 hardware:

- **FP32:** Recorded a generation time of 13.46 seconds.
- **INT8:** Demonstrated a significant latency increase to 27.71 seconds. This is attributed to the "quantization overhead" on the Turing architecture, where the GPU must perform additional casting operations (integer to floating-point) during computation because the T4's INT8 kernels are not as optimized for this specific model's data flow.
- **INT4 (NF4):** Achieved the fastest generation time at 9.63 seconds. The use of Normalized Float 4 (NF4) and optimized *bitsandbytes* CUDA kernels bypassed the bottlenecks seen in INT8, providing a 28.4% speedup over full precision.

Fig. 2 illustrates the inference time for a standardized prompt. While INT4 achieves the fastest performance (9.63s), the INT8 model exhibits an unexpected increase in latency (27.71s). This is primarily due to the computational overhead required for integer-to-float conversions on the Tesla T4 GPU architecture.

C. Accuracy and Perplexity Trade-offs

Perplexity was used as the metric to assess the information loss caused by weight compression.

- **FP32:** Baseline perplexity of 41,381.
- **INT8:** Resulted in a perplexity of 42,282, showing a negligible loss in language modeling capability.
- **INT4:** Interestingly, the 4-bit model recorded a perplexity of 40,599. While lower perplexity usually suggests better performance, in this context of high-compression quantization, it may indicate a slight narrowing of the model's output distribution (reduced variance) rather than a true increase in "knowledge."

D. Comparative Summary

The analysis reveals that while 8-bit quantization is often viewed as a "safe" middle ground, the hardware-specific optimizations for 4-bit NF4 make it the superior choice for the Tesla T4 GPU. The 4-bit configuration provides the best balance, delivering the lowest memory usage and the highest speed without significant degradation in the model's structural integrity.

IX. INTERPRETATION AND DISCUSSION

The experimental results provide a comprehensive view of the trade-offs involved in model compression. This section interprets the relationship between numerical precision, hardware utilization, and model intelligence.

A. The Memory-Precision Trade-off

The reduction of the memory footprint from 4,714 MB (FP32) to 965 MB (INT4) confirms a near-linear saving in storage requirements. This is interpreted as a successful removal of redundant bit-depth. Since the *Orpo-Llama-3.2-1B* model weights typically follow a normal distribution, the use of **NormalFloat 4 (NF4)** quantization effectively captures the most significant information in the distribution while discarding the low-impact decimal noise. This enables the deployment of billion-parameter models on hardware with less than 2GB of available VRAM.

B. Hardware-Specific Latency Anomalies

A key finding in the interpretation of results is the increased latency of the INT8 model (27.71s) compared to the FP32 baseline (13.46s). This phenomenon is interpreted as **Quantization Overhead**. On the Tesla T4 architecture, the computational cost of de-quantizing 8-bit integers back into floating-point values for tensor operations exceeds the time saved by smaller data movement. Conversely, the INT4

model achieved a speedup (9.63s) because the *bitsandbytes* library utilizes highly optimized 4-bit CUDA kernels that minimize these conversion bottlenecks. This suggests that quantization benefits are not universal but are highly dependent on the synergy between the software kernels and the GPU architecture.

C. Structural Integrity of Weights

The inspection of the **First Layer Weights** revealed that while the values were compressed, the relative weight distribution remained stable. This stability explains why the perplexity did not degrade significantly. The fact that INT4 precision maintained a perplexity of 40,599 (compared to FP32’s 41,381) indicates that the “instruction-tuned” nature of the ORPO model provides a level of structural robustness, allowing the model to retain its linguistic logic even when its parameters are represented with only 16 possible discrete values.

D. Conclusion of Analysis

We interpret these findings as a validation of **4-bit NF4** as the optimal configuration for this specific model-hardware pair. While 8-bit quantization is theoretically “safer” for accuracy, the lack of computational speedup on T4 hardware makes it less practical than the 4-bit approach, which offers both maximum memory efficiency and peak inference speed.

X. LIMITATIONS

While quantization offers significant advantages for model deployment, this study identifies several critical limitations that must be considered when applying these techniques to production environments.

A. Quantization Error and Precision Loss

The primary limitation of 4-bit and 8-bit quantization is the inherent loss of numerical precision. Squeezing high-precision 32-bit weights into a much smaller set of discrete values (16 levels for INT4) results in *quantization noise*. Although the perplexity in this study remained stable, extreme compression can lead to “model collapse” or a loss of nuance in complex reasoning tasks. The mapping of large precise numbers to smaller compact numbers inherently discards the “long tail” of the weight distribution, which may contain critical information for edge cases.

B. Hardware Dependency and Latency Overheads

As observed in our results, quantization does not always guarantee faster inference. The increased latency of the INT8 model on the Tesla T4 GPU highlights a significant hardware limitation. Many older or mid-range GPU architectures are not natively optimized for 8-bit integer arithmetic in the context of Transformer layers. Consequently, the overhead required to cast tensors between INT8 and floating-point formats can

outweigh the benefits of reduced memory bandwidth, making the “optimal” quantization level highly dependent on the specific GPU generation.

C. Cost and Training Constraints

This study utilized Post-Training Quantization (PTQ), which is computationally efficient but can be less accurate than Quantization-Aware Training (QAT). Implementing QAT, where the model learns to compensate for precision loss during the training phase, is extremely costly in terms of both time and financial resources. For large-scale models, the infrastructure required to perform high-fidelity quantization training remains a barrier for many researchers and smaller organizations.

D. Scope of Metrics

Finally, this evaluation relied on perplexity as a proxy for accuracy. While perplexity measures the model’s internal confidence, it does not always correlate perfectly with real-world task performance, such as factual accuracy, logical consistency, or safety alignment. A more comprehensive evaluation would require task-specific benchmarks (e.g., MMLU or GSM8K) to fully understand the qualitative impact of 4-bit compression.

XI. FUTURE WORK

The findings of this study open several avenues for further research into Large Language Model (LLM) optimization and deployment. Future efforts will focus on enhancing the precision-efficiency frontier through the following areas:

A. Quantization-Aware Training (QAT)

While this project successfully implemented Post-Training Quantization (PTQ), future work could involve *Quantization-Aware Training*. By simulating quantization noise during the fine-tuning phase, the model can learn to compensate for the loss of numerical precision. This is expected to further stabilize perplexity and maintain higher factual accuracy at 4-bit or even 2-bit levels.

B. Evaluation of 2-bit and Sub-integer Quantization

Recent advancements in extreme compression, such as QuIP and HQQ, suggest that models can remain functional even at 2-bit precision. Extending this study to evaluate sub-integer formats would provide deeper insights into the minimum bit-width required for a 1-billion parameter model to retain its logical reasoning capabilities.

C. Cross-Platform Hardware Benchmarking

The latency anomalies observed on the Tesla T4 GPU suggest that quantization benefits are hardware-dependent. Future research should include a comparative analysis across different GPU architectures (e.g., NVIDIA A100, H100) and edge-specific hardware such as Apple Silicon (M-series) or

specialized AI accelerators (TPUs). This would help establish a "hardware-aware" quantization guide for developers.

D. Deployment in Real-Time Edge Applications

A natural progression of this work is the integration of the INT4 optimized model into a mobile or embedded application. This would involve measuring battery consumption and thermal performance during continuous inference, providing a more holistic view of the environmental impact and sustainability of quantized LLMs.

E. Broadening Evaluation Metrics

Beyond perplexity, future evaluations should incorporate domain-specific benchmarks such as MMLU (Massive Multitask Language Understanding) or HumanEval. This would provide a qualitative assessment of whether 4-bit compression impacts the model's ability to perform specialized tasks like mathematical reasoning or code generation.

XII. CONCLUSION

This project successfully evaluated the impact of 4-bit and 8-bit quantization on the inference efficiency and accuracy of the *Orpo-Llama-3.2-1B-15k* model. Through empirical testing on a Tesla T4 GPU, we have demonstrated that model quantization is an indispensable technique for deploying Large Language Models (LLMs) in resource-constrained environments.

Our findings lead to the following conclusions:

- **Significant Memory Reduction:** Quantization effectively reduced the GPU memory footprint from 4,714 MB (FP32) to 965 MB (INT4), a total saving of nearly 80%. This allows billion-parameter models to reside in less than 1GB of VRAM.
- **Inference Speed Dynamics:** While 4-bit NormalFloat (NF4) achieved the fastest inference time (9.63s), 8-bit quantization suffered from latency overhead (27.71s) due to hardware-specific optimization limits on the Turing architecture. This highlights that the fastest model is not always the one with the highest bit-width.
- **Accuracy Preservation:** Perplexity measurements remained stable across all precision levels. The INT4 model demonstrated that structural robustness can be maintained even at extreme compression levels, making it highly viable for instruction-following tasks.

In summary, while 8-bit quantization provides a theoretically balanced approach, **4-bit NF4 quantization** emerged as the optimal configuration for our specific hardware setup, providing the best combination of maximum memory efficiency and peak computational speed. These results validate the feasibility of scaling state-of-the-art AI to consumer-grade hardware without compromising the model's core linguistic capabilities.

ACKNOWLEDGMENT

The authors would like to thank their faculty guide and the department for their guidance and support for the project.

REFERENCES

- [1] Meta AI, "Llama 3.2: Lightweight Edge-Ready Models," Meta Research, 2024. [Online]. Available: <https://ai.meta.com/blog/llama-3-2-connect-2024/>
- [2] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale," arXiv preprint arXiv:2208.07339, 2022.
- [3] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient Finetuning of Quantized LLMs," arXiv preprint arXiv:2305.14314, 2023.
- [4] T. Wolf et al., "Transformers: State-of-the-art Natural Language Processing," in Proc. of EMNLP, 2020.
- [5] A. Lucek, "Orpo-Llama-3.2-1B-15k Model Card," Hugging Face, 2024. [Online]. Available: <https://huggingface.co/AdamLucek/Orpo-Llama-3.2-1B-15k>

APPENDIX IMPLEMENTATION CODE

Below is the core implementation used in this project to evaluate the memory and inference trade-offs for FP32, INT8, and INT4 precision levels.

```
1 import torch
2 import time
3 import pandas as pd
4 import seaborn as sns
5 import matplotlib.pyplot as plt
6 import numpy as np
7
8 from transformers import (
9     AutoTokenizer,
10     AutoModelForCausalLM,
11     BitsAndBytesConfig
12 )
13
14 model_id = "AdamLucek/Orpo-Llama-3.2-1B-15k"
15
16 # FP32 Model
17 base_model = AutoModelForCausalLM.from_pretrained(
18     model_id,
19     torch_dtype=torch.float32,
20     device_map="auto"
21 )
22
23 print("FP32 Model Loaded")
24
25 #INT8 config
26 bnb_config_8bit = BitsAndBytesConfig(
27     load_in_8bit=True
28 )
29
30 model_8bit = AutoModelForCausalLM.from_pretrained(
31     model_id,
32     quantization_config=bnb_config_8bit,
33     low_cpu_mem_usage=True,
34     device_map="auto"
35 )
36
37 print("INT8 Model Loaded")
38
39 bnb_config_4bit = BitsAndBytesConfig(
40     load_in_4bit=True,
41     bnb_4bit_use_double_quant=True,
42     bnb_4bit_quant_type="nf4",
43     bnb_4bit_compute_dtype=torch.float16
```

```

44 )
45
46 model_4bit = AutoModelForCausalLM.from_pretrained(
47     model_id,
48     quantization_config=bnb_config_4bit,
49     low_cpu_mem_usage=True,
50     device_map="auto"
51 )
52
53 print("INT4 Model Loaded")
54
55 # Loading Tokenizer
56 tokenizer = AutoTokenizer.from_pretrained(model_id)
57
58 # GPU Memory Footprint
59 results = []
60 results.append({
61     "Model": "FP32",
62     "Memory_MB": base_model.get_memory_footprint() /
63         1024**2
64 })
65
66 results.append({
67     "Model": "INT8",
68     "Memory_MB": model_8bit.get_memory_footprint() /
69         1024**2
70 })
71
72 results.append({
73     "Model": "INT4",
74     "Memory_MB": model_4bit.get_memory_footprint() /
75         1024**2
76 })
77
78 print(pd.DataFrame(results))
79
80 # Looking at the full model architecture
81 base_model
82
83 # Weights from the first layer
84 base_weights = base_model.model.layers[0].self_attn.
85     q_proj.weight.data
86 print("original Weights:")
87 print(base_weights)
88 print("Shape: ", base_weights.shape, "\n")
89 print("-" * 50, "\n")
90
91 # Weights from the first layer - 8bit
92 weights_8bit = model_8bit.model.layers[0].self_attn.
93     q_proj.weight.data
94 print("INT8 weights:")
95 print(weights_8bit)
96 print("Shape: ", weights_8bit.shape, "\n")
97 print("-" * 50, "\n")
98
99 # Weights from the first layer - 4bit
100 weights_4bit = model_4bit.model.layers[0].self_attn.
101     q_proj.weight.data
102 print("INT4 weights:")
103 print(weights_4bit)
104 print("Shape: ", weights_4bit.shape, "\n")
105
106 # Function for Generating with the models
107 def generate_response(model, message):
108     # Format msg
109     msg = [{"role": "user", "content": message}]
110     prompt = tokenizer.apply_chat_template(msg,
111         tokenize=False, add_generation_prompt=True)
112
113     # Tokenize and generate
114     inputs = tokenizer(prompt, return_tensors="pt").to(
115         "cuda")
116
117     torch.cuda.synchronize()
118     start = time.time()
119
120     outputs = model.generate(
121         **inputs,
122         max_new_tokens=256,
123         temperature=0.7,
124         do_sample=True,
125         top_k=50,
126         top_p=0.95
127     )
128
129     torch.cuda.synchronize()
130     inference_time = time.time() - start
131
132     # Decode
133     response = tokenizer.decode(
134         outputs[0],
135         skip_special_tokens=True
136     )
137     return response, inference_time
138
139 prompt = "Explain quantization in large language
140 models."
141
142 # FP32
143 base_response, fp32_time = generate_response(
144     base_model, prompt)
145
146 # INT8
147 int8_response, int8_time = generate_response(
148     model_8bit, prompt)
149
150 # INT4
151 int4_response, int4_time = generate_response(
152     model_4bit, prompt)
153
154 print("FP32 Response:\n")
155 print(base_response)
156 print("-" * 50)
157
158 print("\nINT8 Response:\n")
159 print(int8_response)
160 print("-" * 50)
161
162 print("\nINT4 Response:\n")
163 print(int4_response)
164
165 # Calculating Perplexity
166 def calculate_perplexity(model, tokenizer, text):
167     # Encode the txt
168     encodings = tokenizer(text, return_tensors="pt").
169         to("cuda")
170
171     # Define input_ids and target_ids
172     input_ids = encodings.input_ids
173     target_ids = input_ids.clone()
174
175     with torch.no_grad():
176         # Forward pass through the model
177         outputs = model(input_ids, labels=input_ids)
178
179         # Loss calculation
180         loss = outputs.loss
181
182         # Perplexity calculation
183         perplexity = torch.exp(loss)
184
185     return perplexity.item()
186
187 fp32_ppl = calculate_perplexity(base_model,
188     tokenizer, base_response[0])
189 int8_ppl = calculate_perplexity(model_8bit,

```



```

tokenizer, int8_response[0])
178 int4_ppl = calculate_perplexity(model_4bit,
tokenizer, int4_response[0])
179
180 import pandas as pd
181 results_df = pd.DataFrame({
182     "Model": ["FP32", "INT8", "INT4"],
183     "Inference_Time": [
184         fp32_time,
185         int8_time,
186         int4_time
187     ],
188     "Perplexity": [
189         fp32_ppl,
190         int8_ppl,
191         int4_ppl
192     ],
193     "Memory_MB": [
194         base_model.get_memory_footprint()/1024**2,
195         model_8bit.get_memory_footprint()/1024**2,
196         model_4bit.get_memory_footprint()/1024**2
197     ]
198 })
199
200 print(results_df)
201
202 # Plot: Inference Time
203 plt.figure(figsize=(7,5))
204
205 sns.barplot(
206     x="Model",
207     y="Inference_Time",
208     data=results_df
209 )
210
211 plt.title("GPU Inference Time Comparison")
212 plt.ylabel("Time (seconds)")
213 plt.show()
214
215 # Plot: GPU Memory Usage
216 plt.figure(figsize=(7,5))
217
218 sns.barplot(
219     x="Model",
220     y="Memory_MB",
221     data=results_df
222 )
223
224 plt.title("GPU Memory Usage")
225 plt.ylabel("Memory (MB)")
226 plt.show()

```